

CS 218 — Programmable & Interoperable Blockchain

Project Marks Allocation — Detailed Breakdown

Total: 40 Marks per Project (25 Submission + 15 Presentation) · Teams of up to 6 · Presentation: 25–26 April 2026 · Submission Deadline: 10 May 2026, 11:59 PM

How This Document Is Organised

The total 40 marks per project are split as follows: 25 marks for the Project Submission (deadline: 10 May 2026, 11:59 PM) and 15 marks for the Project Presentation (25–26 April 2026). The submission rubric below is marked out of 25. The presentation is graded separately out of 15 — see the Presentation Rubric section below. The submission rubric is identical for all 10 projects. This document translates that rubric into project-specific guidance — for every project, you will find exactly which functions earn which marks, what access control and edge cases are expected, which tests must pass, and what the frontend must demonstrate.

Presentation Rubric — 15 Marks (25–26 April 2026)

Criterion	Max	What Earns the Mark	Marks
Live Demo (Working DApp Demonstration)	6	Smart contract deploys and all core functions execute correctly on-screen (3). MetaMask wallet connects and at least one state-changing transaction is demonstrated live via the DApp frontend (3).	6
Technical Explanation (Design & Architecture)	5	Team clearly explains the contract architecture, key design decisions (on-chain vs off-chain, access control), and security measures taken (2). Gas optimisation approach is presented with before/after numbers (2). On-chain vs off-chain data choices are justified (1).	5
Q&A / Understanding	4	Each team member can answer questions on their own code (2). Team demonstrates understanding of blockchain concepts underpinning their project (reentrancy, gas, IPFS, GDPR) (2).	4
PRESENTATION TOTAL			15

Project Submission Rubric — 25 Marks (Deadline: 10 May 2026, 11:59 PM)

Category	Max	Sub-Criteria	Marks
Smart Contract Correctness	9	Core logic correct (all required functions implemented and behave correctly)	5
		Access control (only authorised addresses can call privileged functions)	2
		Edge cases handled (zero values, expired deadlines, empty arrays, etc.)	2
Security	4	Reentrancy guard on all ETH-sending functions	2
		Input validation — all require() statements; no over/underflow	1

		Visibility correct (public/private/internal/external assigned well)	1
OpenZeppelin Usage	1	OZ contracts used where applicable (Ownable, ReentrancyGuard, token standards)	1
Gas Optimisation	3	Gas report submitted (from hardhat-gas-reporter or forge --gas-report)	1
		One concrete optimisation shown with before/after numbers and explanation	2
Testing	4	Happy-path tests for every core function	2
		Revert/failure tests (unauthorised or invalid calls revert correctly)	1
		Coverage report showing 70% or greater line coverage	1
DApp Frontend	3	Wallet connection (MetaMask, contract ABI loaded, balance displayed)	1
		At least one state-changing function callable from UI with status feedback	2
Documentation & Code	1	README complete (setup steps, team members, roll numbers) + NatSpec comments (@notice, @param) on all public functions	1
SUBMISSION TOTAL			25

Deductions (applied on top of the 25 submission marks above):

- **-2 marks** GitHub repository is private at submission time.
- **-3 marks** Contract does not compile.
- **-1 mark each** Missing team member name in README.
- **-2 marks** No tests are submitted.

The Cardinal Rule: On-Chain vs Off-Chain Data

Blockchain storage is expensive, permanent, and public. Every byte stored on-chain costs real gas and is visible to every node on the network forever. This has direct consequences for cost, privacy, and legal compliance.

Store ON-CHAIN when data...	Keep OFF-CHAIN when data...
Is required by smart contract logic (e.g. bid amount, deadline)	Is a human-readable description, title, or image
Must be verified or enforced without trusting any server	Can be verified by storing only its keccak256 hash on-chain
Is a financial balance or token ownership record	Contains personal details (name, email, ID number, address)
Is an access control record (who is a verifier, who is an owner)	Is subject to GDPR right-to-erasure — immutability prevents deletion
Is a cryptographic commitment (a hash, not the secret itself)	Is large in size — images, PDFs, JSON metadata, documents

The IPFS pattern (mandatory for content data): Store your file or JSON on IPFS, get back a CID (a 46-character content hash). Store ONLY that CID on-chain. Anyone can verify the content by fetching from IPFS and hashing it — the on-chain CID is the proof of authenticity.

Gas cost at a glance: SSTORE (new slot) = 20,000 gas. Each non-zero calldata byte = 16 gas. A 500-character description stored as a string on-chain is approximately 40,000 gas. The same description on IPFS with only a 32-byte CID on-chain is approximately 512 gas. That is a 78x reduction.

GDPR and immutability: Blockchain data is permanent. GDPR Article 17 gives users the right to have their personal data erased. These two facts are irreconcilable if personal data is stored on-chain. The solution is always: store personal data off-chain (encrypted, deletable), store only a non-reversible hash on-chain.

Project 7 — NFT Event Ticketing (ERC-721)

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
createEvent()	1	Stores event struct including maxTickets and royaltyBps
buyTicket() payable	1	Mints ERC-721 token; transfers ETH to organiser; reverts beyond maxTickets
buyResaleTicket() payable	2	Royalty deducted from sale price; seller receives remainder; organiser receives royalty
listForResale() + cancelResaleListing() + royaltyInfo()	1	Holder listing; cancellation; EIP-2981 royaltyInfo returns (receiver, royaltyAmount)
Access Control	2	Only ticket holder can listForResale and cancelResaleListing. Only organiser deploys and creates events.
Edge Cases	2	Buying beyond maxTickets reverts; purchasing cancelled listing reverts; royalty must actually be deducted in buyResaleTicket (not just royaltyInfo).
Sub-total		9 marks

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: buyTicket() and buyResaleTicket() (both transfer ETH).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ERC721URIStorage; IERC2981; ReentrancyGuard on buy functions.

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report. Paste the output table into report.pdf.
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use uint256 instead of uint8 in loops, mark a function pure/view). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
-------	---------------------------------

2	- Buying beyond maxTickets reverts - Non-owner cannot list ticket for resale - Royalty split: 1 ETH resale at 10% → seller 0.9 ETH, organiser 0.1 ETH - ERC-721 ownerOf() updates correctly after each transfer - Cancelled listing cannot be purchased
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing >= 70% line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: buyTicket() — user selects an event and purchases an NFT ticket by paying via MetaMask; display tokenId.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (@notice, @param, @return) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Storing metadata JSON directly on-chain costs thousands of gas per ticket. The correct pattern is tokenURI = 'ipfs://<CID>' where the CID points to the JSON off-chain. NFT ownership (who holds the token) is public — do NOT encode attendee names or seat numbers inside on-chain token data.

GDPR / Privacy: NFT token ownership is permanently public. If ticket holder identity must be verified at the venue, do that off-chain. Never store the attendee's real name or contact details in the NFT metadata on-chain.

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Token ID, event ID, organiser address, owner address	Token metadata JSON (name, description, image URL) — store on IPFS; tokenURI returns the IPFS CID
Royalty basis points (uint96), resale listing price (wei)	Ticket artwork, event poster images — store on IPFS
maxTickets, ticketsSold counter, priceWei per ticket	Attendee personal details (name, email for confirmation) — off-chain ticketing backend only
tokenURI pointing to IPFS CID (a string pointer, not the metadata itself)	Event venue details, schedule, terms — off-chain

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness 9 marks	Security 4 marks	OpenZeppelin 1 marks	Gas 3 marks	Testing 4 marks	Frontend 3 marks	Docs 1 marks
------------------------	---------------------	-------------------------	----------------	--------------------	---------------------	-----------------